

Chapters 6 and 7, Before You Read

The big ideas in plain language, plus what to focus on in two long chapters

Chapters 6 and 7 cover a lot of ground, from the physical cell inside a DIMM to the tradeoffs behind RAID. This document does two things. Part 1 walks through the big ideas in plain language, the same ideas the walkthrough touches live on a real machine, with a little more room to explain the why behind each one. Part 2 narrows the two chapters down to what is worth a close read versus a quick skim, so the length of the chapters does not get in the way of the handful of ideas that matter most for class. In class, several of these ideas get picked up directly on a live Linux machine, so nothing here needs to be memorized ahead of time, just understood well enough to recognize when you see it.

1. The Big Ideas, In Plain Language

The Memory Family Tree

Chapter 6 opens with a family of memory technologies, all built from the same basic idea, a cell that can be written at least once and read many times, and holds one of two states. Table 6.1 sorts that family by a few real distinctions rather than brand names. Volatility separates memory that forgets everything the instant power is removed, DRAM and SRAM, from memory that remembers with or without power, the ROM family. Within volatile memory, DRAM and SRAM make a genuine tradeoff. DRAM stores a bit as a tiny electrical charge on a capacitor, which leaks over time, so it needs to be refreshed thousands of times a second just to remember what it is holding, and in exchange a DRAM cell is extremely small and cheap, which is why main memory is built from DRAM. SRAM stores a bit using several transistors wired to hold their own state, so it needs no refreshing and is far faster, and in exchange it takes more silicon per bit and costs more, which is why SRAM is reserved for a small amount of very fast cache rather than gigabytes of main memory. Read only memory is not one technology but a spectrum, from memory permanently burned in at the factory to memory that can be reprogrammed in place, and the difference between PROM, EPROM, and EEPROM is really a difference in how, and how many times, it can be changed after it leaves the factory.

The Price of Trusting Memory

Every memory cell can, on rare occasion, flip a bit for no software reason at all, a stray charge, cosmic radiation, or simple wear. Error correction is the answer to that problem, and Chapter 6 walks through the Hamming code as the classic way to do it. The idea only makes sense once you see the actual cost, which is exactly what Table 6.2 shows, adding single bit error correction to a word does not cost one extra bit, it costs several, and extending that to catch two bit errors while correcting one costs several more. That is a genuine tradeoff, not a free feature, which is why most consumer memory skips it entirely and most server memory pays for it. The mechanism itself, laying out data bits and check bits so that a handful of parity checks can point at exactly which bit is wrong, is worth understanding at the level of the diagram rather than the arithmetic, since that same idea reappears constantly in computing, from memory to network packets to disk sectors.

Memory Keeps Getting Faster, the DDR Story

The second half of Chapter 6's DRAM material traces one continuous idea, getting more data across the same pins on every clock cycle. Synchronous DRAM ties memory operations to the same clock the rest of the system uses, so the memory controller can pipeline requests instead of waiting on each one individually. Double data rate DRAM pushes that further by transferring data on both the rising and falling edge of the clock, doubling the effective rate without doubling the clock speed itself. Table 6.4's progression from DDR1 through DDR4 is really the same trick applied again and again, higher clock rates and wider prefetch buffers, not a new idea each generation. The specific numbers in that table matter less than recognizing the pattern, and being able to say, given a DDR generation, roughly how it compares to its neighbors.

Flash Memory, the Technology That Sits on the Line

Flash memory closes out Chapter 6, and it is worth pausing on, since it does not fit neatly into the internal versus external memory split the book uses to divide Chapters 6 and 7. Flash is non-volatile like ROM, but it can be erased and rewritten many times like RAM, which is why it shows up both as internal memory, in embedded and mobile devices, and as external memory, in the SSDs that Chapter 7 covers as a rival to the spinning disk. Keep that dual identity in mind, since it is the bridge between what otherwise reads like two separate chapters.

What Actually Makes One Storage Technology Different From Another

Chapter 7 opens external memory with the same kind of comparison Chapter 6 used for internal memory, but the deciding factor this time is mechanism. A magnetic disk stores data as magnetized regions on a spinning platter, and reading or writing means physically moving a head to the right track and waiting for the right sector to rotate underneath it. A solid state drive stores data electrically, in flash cells, with no moving parts and no wait for anything to spin into position. That single mechanical difference explains nearly everything else the chapter compares between the two, and it is the exact distinction a live Linux system reports as one column, whether a device is rotational or not.

Why Anyone Would Want More Than One Disk

RAID exists because a single disk forces a tradeoff between speed, capacity, and safety that nobody actually wants to make. Table 7.3 lays out several ways to combine multiple disks into one logical unit, each choosing a different point in that tradeoff. Striping data across disks, Level 0, buys speed by spreading the work, at the cost of making the whole array more fragile than a single disk, since losing any one member loses everything. Mirroring, Level 1, buys safety by writing every byte twice, at the cost of using twice the disks for the same capacity. The higher levels layer parity information on top of striping to buy some of both, at the cost of real computation on every write. No one level is simply better than the others, each is a specific answer to the question of what you are willing to spend more of, and what you refuse to risk losing.

Solid State's Own Quirks

Chapter 7 is careful to note that SSDs are not simply faster disks, they behave differently in ways that matter. A flash cell wears out after a limited number of erase cycles, so an SSD's controller constantly moves data around to spread writes evenly across the whole drive, a process called wear

leveling, invisible from the outside but the reason SSD performance can change over the life of the drive. That same difference in mechanism is also why Linux treats SSDs and spinning disks differently at the software level, assigning them different I/O schedulers, which is one of the things class looks at directly.

2. What to Look for in the Book

Several of the live Linux commands in class map directly onto a specific table or figure in these two chapters. Finding these ahead of time in your own reading will make the live version click faster, since you will already know what you are looking at.

Table 1: Where each live command comes from in the text.

Concept	What to look for	Textbook location
DIMM inventory	Table 6.1's family of semiconductor memory types, and Figure 6.2's diagram of how a DRAM cell differs physically from an SRAM cell	Ch 6, Table 6.1, Fig 6.2
Error correction type	Table 6.2, showing exactly how many extra bits a word must carry to gain single-bit or double-bit error correction	Ch 6, Table 6.2
Live ECC error counts	Figures 6.9 to 6.11, the bit-by-bit mechanics of how a Hamming code actually computes and checks its check bits	Ch 6, Fig 6.9 to 6.11
Rotational versus solid state	Figure 7.2's diagram of a magnetic disk's physical layout, next to Figure 7.8's diagram of an SSD's architecture with no moving parts at all	Ch 7, Fig 7.2, Fig 7.8
I/O scheduler choice	The seek time, rotational delay, and transfer time that make up disk access time, in the section around Table 7.2	Ch 7, disk performance parameters
RAID Level 1 mirror	Table 7.3's comparison of every RAID level, and Figure 7.6's diagram of Level 1 specifically, drawn as a full duplicate across two disks	Ch 7, Table 7.3, Fig 7.6

3. Reading Guide

Chapter 6, Read Closely

- Figure 6.1, memory cell operation, and Table 6.1's family of semiconductor memory types.
- Dynamic RAM, Static RAM, and the SRAM versus DRAM comparison.
- Read Only Memory, Programmable ROM, and Read-Mostly Memory.
- Error Correction, Table 6.2, and Figures 6.7 and 6.8, the concept and the cost of Hamming

codes.

- Flash Memory.

Chapter 6, Skim for the Idea

- Figures 6.3 through 6.6, and 6.9 through 6.11, the address organization and bit layout diagrams. Enough to recognize the shape, not to redo the arithmetic.
- Advanced DRAM Organization, Synchronous DRAM, and Double Data Rate SDRAM, Table 6.4, and Figure 6.14. Enough to place a DDR generation in the progression.
- Nonvolatile RAM within the Memory Hierarchy, and Nonvolatile RAM Technologies.

Chapter 6, Skip for Now

- Table 6.3, SDRAM pin assignments, and the SDRAM timing diagram.
- Embedded DRAM, and its use in the IBM z13 and Intel Core chip layouts, Figures 6.15 and 6.16.
- Kiviat graphs for flash memory, Figure 6.19.

Chapter 7, Read Closely

- Magnetic Disk, and Magnetic Read and Write Mechanisms.
- Disk Performance Parameters and Table 7.2.
- RAID, Table 7.3, Figures 7.6 and 7.7, and RAID Levels 0 and 1 specifically.
- SSD Compared to HDD, Table 7.5, Figure 7.8, and Practical Issues.

Chapter 7, Skim for the Idea

- Disk Data Layout and the comparison of disk layout methods, Figures 7.2 and 7.3.
- Disk Classification.
- RAID Levels 2 through 6, and Table 7.4's comparison. Enough to place each level relative to Level 0 and Level 1.

Chapter 7, Skip for Now

- Legacy and Advanced Sector Formats, Figure 7.4, and Table 7.1's physical characteristics.
- Everything on optical disks, CD-ROM, CD-R, CD-RW, DVD, and Blu-ray.
- Everything on magnetic tape, including Table 7.7's LTO comparison.